Comparative Analysis of Transfer Learning Strategies on Caltech-101 Using Pre-Trained CNN Architectures

1st Adithya S*, 2nd Amrit Gopinath*, 3rd Harshil Malisetty*
*Department of CSE, Sri SivaSubramanyia Nadar College of Engineering, Chennai, India
adithya2410422@ssn.edu.in
amrit2410182@ssn.edu.in
harshil2410093@ssn.edu.in

***Abstract*—Object recognition is a key task in image processing and computer vision. Deep learning methods, in particular transfer learning, have enabled leveraging powerful models despite very little training data.**

In the present paper, pre-trained CNN models were employed to develop an object classifier for the Caltech-101 dataset. In order to test and compare their performance in real applications, the architectures MobileNetV2, EfficientNetB2, ResNet, and DenseNet were evaluated.

In this study, an end-to-end deep learning pipeline was deployed, comprising dataset curation, data preprocessing, pre-trained model loading, feature extraction, fine-tuning, and ultimate evaluation. One of our primary objectives was to compare the performance of models with frozen and unfrozen (fine-tuned) layers. Such comparisons help to explain the difference between these two in terms of training time and accuracy.

Model complexity, computational cost, and joint performance have been studied in this paper. While MobileNet is cheap in terms of hardware resource usage, Residual Network (ResNet), DenseNet, and recently the EfficientNet could be used for better feature extraction with an additional computation budget. For illustration purposes, our paper aims to stress where models are chosen under different resources and transfer learning in the wild.

***Index Terms*—Transfer Learning, Object Recognition, Caltech-101, Convolutional Neural Networks, MobileNet, EfficientNet, ResNet, DenseNet, Feature Extraction, Fine-Tuning, Deep Learning, Image Classification, Comparative Analysis.**

I. Introduction

CNN has become the de-facto standard for image classification in recent years and obtained remarkable performance, benefiting from its ability to automatically learn and abstract critical features of raw images. Further, it should be noted that Deep Learning (DL) is quite effective for solving image related problems, including face recognition. The dynamic CNNs is an increasingly rich field and includes not only the traditional but also modern architectures, which may serve as desirable testbed for evaluating these methods in relation to their relative performance on a variety of datasets. This justifies the usage and dominance of CNNs.

979-8-3315-6432-2/26/$31.00 © 2026 IEEE

Machine learning models are commonly designed to resolve specific tasks and work in isolation. In these cases, the model should be constructed and trained from scratch and requires a lot of data. Depending on the kind of problem to be solved or the type of model, millions of samples may be required. This highlights the cost of training on millions of images and GPU requirements. Abundant data and computing now empower researchers and companies to try solving complex problems that were not possible earlier. One such category is Computer Vision (CV) problems which deal with image processing.

Data volume is critical in Computer Vision problems using Machine Learning (ML), where an ML model learns better with more images. However, obtaining training data can be difficult and expensive for certain CV problem domains. Thus, there is an effort to reuse a trained ML model in one Computer Vision domain and apply it to a related CV domain. This effort of re-utilizing a model trained for one use case and applying it to another use case is called Transfer Learning. It is a method used to transfer knowledge acquired from one task to resolve another. A different task solved with a similar problem can be a starting point to extract learned information and fine-tune the model, reducing the isolation paradigm of having two different problems.

Commonly, the initial layers of the pre-trained model are used for feature extraction, while the later layers, which are task-specific, are fine-tuned on the new dataset. This approach is particularly useful in applications where labeled data is scarce or expensive to collect.

Comparing the performances of CNN architectures, has now become of paramount importance in order to know which is better or worse in specific applications. Prior works evaluated their design by benchmark datasets. For instance, Krizhevsky et al. evaluated several CNN models on the ImageNet data set, showing that deep networks can be successful at large-scale image classification (Krizhevsky et al., 2012). Tan and Le also recently developed EfficientNet, which achieved higher performance at low computational cost (Tan and Le, 2019).

The decision of how many layers to fine-tune depends on factors such as the size of the target dataset and the

similarity between the source and target domains. When the target dataset is small or closely related to the source dataset, freezing most of the pretrained layers and training only the final classification layer can be sufficient. In contrast, when the target task differs significantly, unfreezing and fine-tuning deeper layers allows the network to adapt high-level feature representations to the new domain.

Early layers of CNNs generally learn fundamental visual patterns such as edges, textures, and simple shapes, which are broadly applicable across different tasks. These features are considered generic and are often preserved by freezing the corresponding layers during transfer learning. Deeper layers, however, capture more complex and task-specific patterns, such as object parts and semantic structures. Unfreezing and fine-tuning these later layers enables the model to better align with the characteristics of the target dataset.

A common strategy is to begin fine-tuning from the final layers by selectively unfreezing them and progressively including earlier layers based on validation performance. This approach balances model adaptability with generalization, preventing overfitting while improving classification accuracy. By tuning an optimal number of layers through controlled freezing and unfreezing, transfer learning can significantly enhance performance on target tasks without the need for extensive training data or computational resources.

II. Literature Survey

Deep convolutional neural networks have led to a series of breakthroughs for image classification [1], [2]. Deep networks integrate low, mid, and high-level features in an end-to-end multi-layer fashion, and these feature levels can be enriched by increasing network depth. Recent evidence shows that depth is crucial, and leading results on the ImageNet dataset exploit very deep models [**?**], [3]. However, increasing depth introduces challenges such as vanishing and exploding gradients, which hinder convergence. This problem has been addressed by normalized initialization and intermediate normalization layers, enabling networks with tens of layers to converge.

However, as networks have gone deeper and higher accuracy was required, these kind of problems appeared: as network depth increases, the accuracy gets saturated (which might be seen also as a vanishing gradient problem and degradation in the optimization process emerged. This generalization error is not due to overfitting, and actually, for more layers, risk obtained during training is higher. Deep Residual Learning architectures address this issue by introducing shortcut connections which allow for identity mapping to be preserved so that stacked layers can learn residual mappings [3]. These shortcut connections do not add extra parameters or computational complexity; they make the training of very deep networks becomes easier.

The mammalian visual system is pseudo-hierarchical, while the deep hierarchy neural networks are one of the most successful building blocks for object recognition [**?**]. These are the systems that take input images and make decisions based on options. On the other hand, CNN features are learned with supervised learning during backpropagation [2]. Some of the most recently available implementations on GPUs demonstrated these networks to be trainable in days (as opposed to months), opening up a new possibility for researchers to explore architectural parameters and conduct method-based comparison using data-driven benchmarks such as MNIST, NORB, and CIFAR-10 [1].

It is a widely used practice to scale-up ConvNets for getting better accuracy. Ordinary scaling boosts depth, width, or image resolution uniformly, which usually needs manually regulating and meanwhile decreases efficiency. A compound scaling method is used to uniformly scale network width, depth, and resolution for better performance and efficiency [4]. EfficientNet models demonstrate that balanced scaling outperforms conventional methods, achieving state-of-the-art performance with fewer parameters and faster inference. These models also transfer well to multiple datasets while maintaining high accuracy.

Network architectures measured on ImageNet have fueled progress across computer vision tasks, including transfer to new datasets [5]. Transfer learning performance is highly correlated with ImageNet accuracy for fixed features and fine-tuning from ImageNet initialization. Better ImageNet networks provide better penultimate layer features and improved performance when fine-tuned. Architectures transfer well across tasks even when weights do not, highlighting the importance of architecture design in transfer learning.

Transfer learning reduces the demand for large computational resources and training data while providing strong performance on small datasets [6]. Typically, the classification layer of a pre-trained CNN is replaced with a new softmax layer trained on the target dataset. The number of layers to be fine-tuned depends on the size of the target dataset and the similarity between source and target domains. Initial layers represent primitive features such as edges and blobs that are generic to many tasks, while final layers represent task-specific features. Layers can be tuned from final to initial layers based on validation performance, leading to improved classification performance.

CNNs are extensively used to deal with visual inputs; they are based on the processing analogues of the mammalian visual cortex. They require no additional pre-processing and are popularly used in several big-scale competitions such as the ImageNet Large Scale Visual Recognition Challenge [7]. Several pre-trained models for deep image classification such as AlexNet and GoogleNet have obtained remarkable accuracy on ImageNet and are widely adopted for datasets like Caltech-101 [**?**], [1].

III. Methodology

A. Dataset Organization

The classic Caltech-101 dataset consisted of 101 different classes of images, and each class had a variable number of images ranging from 40 to 200, totaling approximately 9,000 images. The use of pre-trained classic CNN models like DenseNet, ResNet is justifiable as the number of classes

present in the dataset is large and the number of images per class is less.

The Dataset was split into three parts, 70% of it was part of training set, 10% was used for validation, and the remaining 20% was used for testing purposes. This was done using a random split and to ensure fairness for each class, the split ratio per class was uniform i.e., 70% of each class was used for training and so on.

A random seed was set to ensure that the results are reproducible, and the split of the dataset is uniform if the experiment is conducted at different locations. This is also to ensure that the results stay consistent and agnostic to the working environment.

B. Pre-processing and Model Details

1) Image Resizing and Normalization: Basic preprocessing was performed on the data, including changing the dimensions of the images to fit the model (224x224 for every model except EfficientNet-b2 where 260x260 was used). Each number in the tensor representation of the images, with the help of toTensor(), was converted from [0-255] to [0-1], and resulting tensor was normalized using the mean and standard deviation of a normal distribution. Data augmentation was purposefully not included to analyze the performance of the established models without using large amount of preprocessing.

2) Model Details and Inputs: The performance of 4 different models was analysed whose details are given in the table below.

TABLE I
MODEL ARCHITECTURE DETAILS

Model	Depth (Layers)	Parameters	Input Size
ResNet18	18	∼11.7M	224x224
DenseNet121	121	∼8.0M	224x224
MobileNetV2	53	∼3.5M	224x224
EfficientNet-B2	140	∼9.2M	260x260

D. Proposed Architecture

The overall workflow of the proposed transfer learning system is illustrated in Fig. 1.

The pipeline includes dataset preprocessing, feature extraction using pre-trained CNN models, fine-tuning of selected layers, and final evaluation on the Caltech-101 dataset.

C. Model Fine-tuning

Each model was trained twice: once with all its layers frozen and trained only the classification head, and the other with the last one or two layers unfrozen (depending on the depth of the model). Each run consisted of an Adam optimizer with learning rate set to $1x10^{-3}$ for fully frozen model or $1x10^{-4}$ for last layer(s) finetuned model. A fixed 8 epochs and batch size of 32 was employed for each run.

Accuracy of the model with validation was calculated for each epoch along with the loss. The models were then tested on the test set and the chosen metrics were accuracy and Macro f1 score which is the average of f1 score taken across all the classes. Macro f1 was chosen as there is a class imbalance and equal importance has to be given for all the classes so that the class with large amount of images would not affect the metrics.

IV. RESULTS AND DISCUSSION

The performance of the proposed transfer learning concept was evaluated using several pre-trained convolution neural network architectures, including EfficientNet-B2, DenseNet-101, EfficientNet-B2, and ResNet-18. The evaluation was conducted on both the fine-tuned and frozen configurations of the model, and the results of the performance are consolidated in the table.

TABLE II
COMPARISON OF CNN ARCHITECTURES ON TEST SET

Model	Test Accuracy (%)	Macro F1 Score
DenseNet-121 (Frozen)	89.24	0.8433
DenseNet-121 (Fine-tuned)	94.70	0.9235
EfficientNet-B2 (Frozen)	92.00	0.8902
EfficientNet-B2 (Fine-tuned)	94.43	0.9186
ResNet-18 (Frozen)	90.56	0.8689
ResNet-18 (Fine-tuned)	95.36	0.9358
MobileNetV2 (Frozen)	90.11	0.8649
MobileNetV2 (Fine-tuned)	93.16	0.9029

As we can infer from the table fig10.1, results obtained from the fine-tuned models are significantly higher than those obtained from the frozen model architecture, for example, DenseNet-101 improved its 89.24% accuracy (in frozen mode) to 94.70% after fine-tuning. At the same time, the same shift is noted on other models like EfficientNet-B2, EfficientNet-B2. This indicates that fine-tuning enhances the formation of the feature vectors and weights associated with it, which in turn increases classification performance.

Similarly, the EfficientNet-B2 CNN model achieved a accuracy of 92.00% in frozen mode, and an increase to 94.43 is noted after fine-tuning along with a corresponding increase in macro F1 from 0.8902 to 0.9186. This consistent increase in the accuracy and the F1 score tells us that fine-tuning reduces the misclassifications made by the model.

Out of all the evaluated architectures, the DenseNet-101 fine-tuned model achieved the highest accuracy of 94.70% in the test set with a macro F1 of 0.9235. This result tells us that the densely connected to the convolution block network enables efficient feature reuse and improved gradient propagation across all the different layers.

The relatively close values between the accuracy and the macro F1 score across all the models indicate that the class imbalance effects are limited in the given dataset.

V. CONCLUSION

In this paper, my team and I individually trained and tested different models on the Caltech-101 dataset using various CNN architectures, namely MobileNet, EfficientNet, ResNet, and DenseNet. We followed the appropriate deep learning workflow, more specifically a transfer learning approach, and

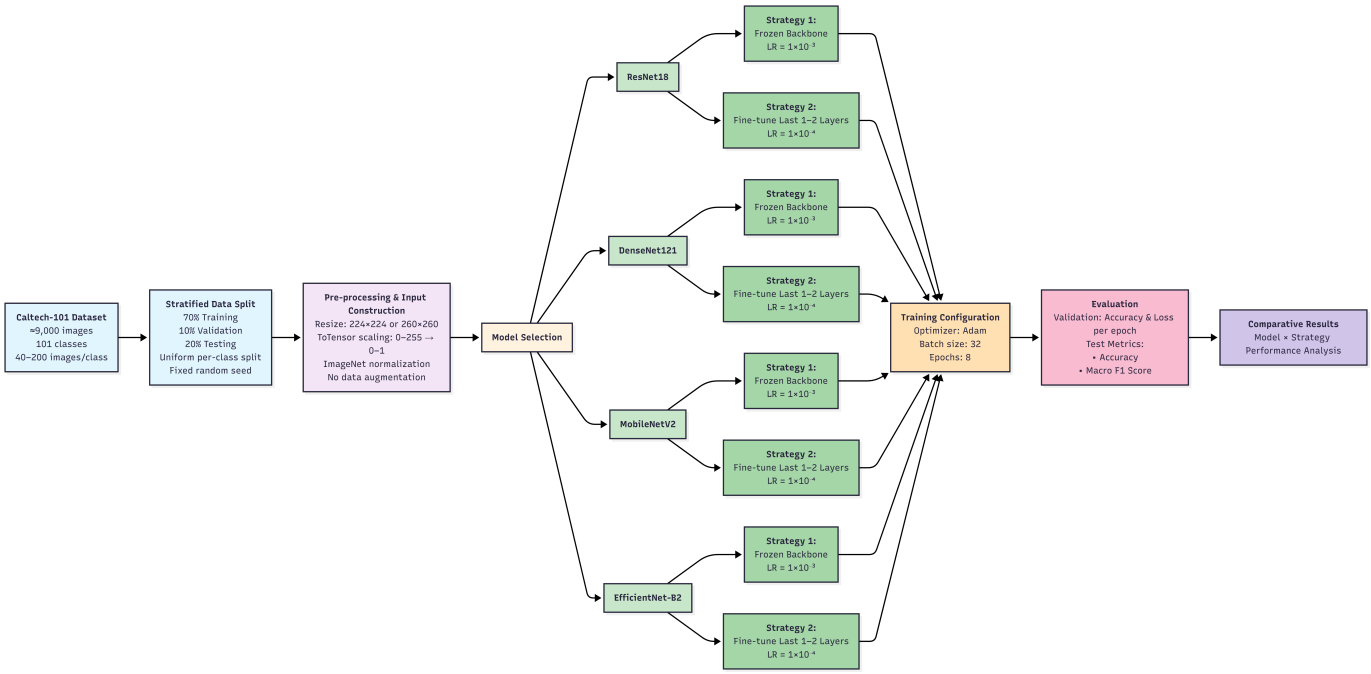

Fig. 1. Transfer learning pipeline using pre-trained CNN architectures for Caltech-101 classification.

obtained two results from each model: one with frozen layers and one with fine-tuning, i.e., unfreezing the last layer, including the convolutional layer and the penultimate layer.

By comparing the models using this metric, it became clear that freezing produces slightly lower accuracy compared to the fine-tuned version, where the penultimate layer is unfrozen. We compared the four models using the macro F1 score and observed that the best macro F1 score was achieved by ResNet-18, with approximately 0.86 (frozen) and 0.93 (fine-tuned). This was followed by EfficientNet with 0.89 and 0.91, and DenseNet with 0.84 and 0.92. MobileNet showed results of 0.86 and 0.9, which remains an outlier in the data. This is due to early stopping during model training when frozen. The implementation of the proposed transfer learning pipeline, including training scripts and evaluation code, is publicly available at: https://github.com/Amrit828/Comparative-Study-of-Pretrained-CNNs-on-Caltech-101.

References

[1] A. Krizhevsky, I. Sutskever, and G. Hinton, "Imagenet classification with deep convolutional neural networks," in *NeurIPS*, 2012.

[2] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, 1998.

[3] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," *CVPR*, 2016.

[4] M. Tan and Q. Le, "Efficientnet: Rethinking model scaling for cnns," *ICML*, 2019.

[5] S. Kornblith, J. Shlens, and Q. Le, "Do better imagenet models transfer better?" in *CVPR*, 2019.

[6] J. Yosinski, J. Clune, Y. Bengio, and H. Lipson, "How transferable are features in deep neural networks?" in *NeurIPS*, 2014.

[7] J. Deng *et al.*, "Imagenet: A large-scale hierarchical image database," *CVPR*, 2009.